

1 What should I submit, where should I submit and by when?

Your submission for this major assignment will be one PDF (.pdf) file and one ZIP (.zip) file. Instructions on how to prepare and submit these files are given below.

Major Assignment Package:

<https://www.cse.iitk.ac.in/users/purushot/courses/ml/2025-26-s/material/assignments/assn1.zip>

Deadline for all submissions: June 12, 2026, 9:59PM IST

Code Validation Script: <https://colab.research.google.com/drive/107xaxbukR0KPFmV25F9TRzfkNn0fLYrk?usp=sharing>

Code Submission: <https://forms.gle/AhrfJWumwfB39R8u8>

Report Submission: on Gradescope

There is no provision for “late submission” for this major assignment

1.1 How to submit the PDF report file

1. The PDF file must be submitted using Gradescope in the *group submission mode*.
2. Unregistered auditors cannot make submissions to this major assignment.
3. Make only one submission per assignment group on Gradescope, not one submission per student. Gradescope allows you to submit in groups - please use this feature to make a group submission.
4. **Ensure that you validate your submission files on Google Colab before making your submission** (validation details below). Submissions that fail to work with our automatic judge since they were not validated will incur penalties.
5. Link all group members in your group submission. If you miss out on a group member while submitting, Gradescope will think that person never submitted anything.
6. You may overwrite your group’s submission as many times as you want before the deadline (submitting again on Gradescope simply overwrites the old submission).
7. Do not submit Microsoft Word or text files. Prepare your report in PDF using the style file we have provided (instructions on formatting given later).

1.2 How to submit the code ZIP file

1. Your ZIP file should contain a single Python (.py) file and nothing else. The reason we are asking you to ZIP that single Python file is so that you can password protect the ZIP file. Doing this safeguards you since even after you upload your ZIP file to your website,

no one can download that ZIP file and see your solution (since you will tell the password only to the instructor). If you upload a naked Python file to your website, someone else may guess the location where you have uploaded your file and steal it and you may get charged with plagiarism later.

2. We do not care what you name your ZIP file but the (single) Python file sitting inside the ZIP file must be named “submit.py”. There should be no sub-directories inside the ZIP file – just a single file. We will look for a single Python (.py) file called “submit.py” inside the ZIP file and delete everything else present inside the ZIP file.
3. Do not submit Jupyter notebooks or files in other languages such as C/C++/R/Julia/Matlab/Java. We will use an automated judge to evaluate your code which will not run code in other formats or other languages (submissions in other languages will get a zero score).
4. Password protect your ZIP file using a password with 8-10 characters. Use only alphanumeric characters (a-z A-Z 0-9) in your password. Do not use special characters, punctuation marks, whitespaces etc in your password. Specify the file name properly in the Google form.
5. Remember, your file is not under attack from hackers with access to supercomputers. This is just an added security measure so that even if someone guesses your submission URL, they cannot see your code immediately. A length 10 alphanumeric password (that does not use dictionary phrases and is generated randomly e.g. 2x4kPh02V9) provides you with more than 55 bits of security. It would take more than 1 million years to go through all $> 2^{55}$ combinations at 1K combinations per second.
6. Make sure that the ZIP file does indeed unzip when used with that password (try `unzip -P your-password file.zip` on Linux platforms).
7. Upload the password protected ZIP file to your IITK (CC or CSE) website (for CC, log on to `webhome.cc.iitk.ac.in`, for CSE, log on to `turing.cse.iitk.ac.in`).
8. Fill in the following Google form to tell us the exact path to the file as well as the password <https://forms.gle/AhrfJWumwfB39R8u8>
9. **Do not host your ZIP submission file on file-sharing services like Dropbox or Google drive. Host it on IITK servers only.** We will autodownload your submissions and GitHub, Dropbox and Google Drive servers often send us an HTML page (instead of your submission) when we try to download your file. Thus, it is best to host your code submission file locally on IITK servers.
10. While filling in the form, you have to provide us with the password to your ZIP file in a designated area. Write just the password in that area. For example, do not write “Password: helloworld” in that area if your password is “helloworld”. Instead, simply write “helloworld” (without the quotes) in that area. Remember that your password should contain only alphabets and numerals, no spaces, special or punctuation characters.
11. While filling the form, give the complete URL to the file, not just to the directory that contains that file. The URL should contain the filename as well.

(a) Example of a proper URL:

`https://web.cse.iitk.ac.in/users/purushot/mlproj/my_submit.zip`

- (b) Example of an improper URL (file name missing):
`https://web.cse.iitk.ac.in/users/purushot/mlproj/`
 - (c) Example of an improper URL (incomplete path):
`https://web.cse.iitk.ac.in/users/purushot/`
12. We will use an automated script to download all your files. If your URL is malformed or incomplete, or if you have hosted the file outside IITK in a manner that is difficult to download, then your group may lose marks.
 13. Make sure you fill-in the Google form with your file link before the deadline. We will close the form at the deadline.
 14. Make sure that your ZIP file is actually available at the link at the time of the deadline. We will run a script to automatically download these files after the deadline is over. If your file is missing, we will treat this as a blank submission.
 15. We will entertain no submissions over email, Piazza etc. All submissions must take place before the stipulated deadline over the Gradescope and the Google form. The PDF file must be submitted on Gradescope at or before the deadline and the ZIP file must be available at the link specified on the Google form at or before the deadline.

Problem 1.1 (Hamming PUF). Frustrated with ML attacks, Melbo decided to go for a low-level but high-tech PUF design. A state-of-the-art TPM cryptoprocessor is used to store a 32-bit secret word $\mathbf{s} \in \{0,1\}^{32}$. Given a 32-bit challenge $\mathbf{c} \in \{0,1\}^{32}$, Melbo first calculates the bit-wise XOR of the challenge with the secret to get a 32-bit word $\mathbf{m} \in \{0,1\}^{32}$ as $\mathbf{m} \stackrel{\text{def}}{=} \mathbf{c} \oplus \mathbf{s}$ (for any two bits $x, y \in \{0,1\}$, we have $x \oplus y = 1$ if $x \neq y$ else $x \oplus y = 0$). The Hamming weight of a word is simply the sum of all its coordinates (where the sum is the usual integer sum, not sum modulo 2). Melbo calculates the *partial Hamming weights* of $\mathbf{m}$. Specifically, the Hamming weight of all even locations is calculated as $h_e \stackrel{\text{def}}{=} m_0 + m_2 + \dots + m_{30}$. Note that $0 \leq h_e \leq 16$. Similarly, the Hamming weight of all odd locations is calculated as $h_o \stackrel{\text{def}}{=} m_1 + m_3 + \dots + m_{31}$. Note that $0 \leq h_o \leq 16$. Melbo then takes the product $h_e \cdot h_o$. If the product is greater than a secret threshold t i.e. $h_e \cdot h_o \geq t$, the response is 1 else the response is 0.

Melbo thinks that the inclusion of low-level operations such as partial Hamming weights and products should make it difficult for a linear model to can predict the responses if given a few thousand challenge-response pairs. Your job is to prove Melbo wrong! You will do this by showing that there do exist linear models that can perfectly predict the responses of a Hamming PUF and these linear models can be estimated fairly accurately if given enough challenge-response pairs (CRPs).

Your Data. We have provided you with data from a Hamming PUF with 32-bit challenges. The training set consists of 7500 CRPs and the test set consists of 2500 CRPs. For each CRP, a 32-bit challenge and a 1-bit response is provided. Each row in the training and testing files provided to you contain 33 bits, each number being 0 or 1. The first 32 numbers correspond to the challenge bits and the last number corresponds to the response bit. If you wish, you may create (held out/k-fold) validation sets out of this data in any way you like. Your job is to create a new feature vector from the 32-bit challenge so that you can learn a linear model to predict the Hamming PUF response very accurately. However, note that unlike the arbiter PUF case where the new feature vector was also 32-dimensional for 32-dimensional challenges, for a Hamming PUF, the new feature vector may have a dimensionality different from that of the challenges.

(150 marks)

Your Task. The following enumerates 5 parts to the question. Parts 1, 2, 4, 5 need to be answered in the PDF report file. Part 3 needs to be answered in the Python file.

1. Give a detailed mathematical derivation (as given in the lecture slides) how a single linear model can predict the responses of a Hamming PUF. Specifically, give an explicit map $\phi : \{0, 1\}^{32} \rightarrow \mathbb{R}^D$ and a corresponding linear model $\mathbf{w} \in \mathbb{R}^D, b \in \mathbb{R}$ that predicts the responses i.e. for all CRPs $\mathbf{c} \in \{0, 1\}^{32}$, we have $\frac{1 + \text{sign}(\mathbf{w}^\top \phi(\mathbf{c}) + b)}{2} = r(\mathbf{c})$ where $r(\mathbf{c})$ is the response of the Hamming PUF on the challenge $\mathbf{c}$. Note that $\mathbf{w}, b$ may depend on the PUF-specific constants such as the secret word $\mathbf{s}$ and the threshold t . However, the map $\phi(\mathbf{c})$ must depend only on $\mathbf{c}$ (and perhaps universal constants such as $2, \sqrt{2}$ etc). The map ϕ must not use PUF-specific constants such as $\mathbf{s}$ or t . (20 marks)
2. What dimensionality D does the linear model need to exactly predict the responses for a Hamming PUF? Give calculations showing how you arrived at that dimensionality. The dimensionality should be stated clearly and separately in your report, and not be implicit or hidden away in some calculations. Maps with excessively large dimensionality, e.g. exponential in bit-length, will lose marks even if they may solve the problem. (10 marks)
3. Write code to implement your map ϕ and provide suitable hyperparameters for the LinearSVC model. Submit code in `submit.py` implementing at least 2 methods (80 marks)
 - (a) `my_map()` that should take raw challenges and map each one of them to a D -dimensional feature vector using the ϕ function that you have proposed.
 - (b) `my_params()` that should return suitable hyperparameter values for the LinearSVC model. Please note the following details about providing hyperparameter values:
 - The hyperparameters must be provided as a Python dictionary that must be compatible with the `set_params()` method of the LinearSVC class in sklearn. Please ensure there are no incompatibilities by checking that the Google Colab validation code does not throw any errors on your submission.
 - LinearSVC has around a dozen hyperparameters. You need not provide a value for every single hyperparameter, just the ones you feel are important. Some of them, e.g. `multi_class` are not even applicable to this problem. The hyperparameters for which you do not provide a value will be set to their default value. Please refer to the API for `sklearn.svm.LinearSVC` for the default values (link: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html>).
 - The original and mapped train features and train labels (for the secret dataset) will be supplied to the `my_params()` method in case you wish to perform hyperparameter turning via validation on the train set. However, it is not mandatory to do so. You may ignore these inputs and provide fixed/preset values of the hyperparameters that you found to work well on the public train/test set. Doing so might get you more marks for speed. The public and secret datasets were generated similarly but it is upto you to decide whether you wish to get more marks for speed or reduce risk and tune the hyperparameters on the secret dataset.

Note that both methods `my_map()`, `my_params()` must be provided in the same `submit.py` file and not in separate files.

4. Report outcomes of experiments (on the public dataset) with both the `sklearn.svm.LinearSVC` and `sklearn.linear_model.LogisticRegression` methods when used to learn the linear model

for breaking the Hamming PUF. In particular, report how various hyperparameters affected training time and test accuracy (on the public dataset) using tables and/or charts. In particular, you must report how at least 2 of the following affect training time and test accuracy:

- (a) changing the `loss` hyperparameter in LinearSVC (hinge vs squared hinge)
- (b) setting `C` in LinearSVC and LogisticRegression to high/low/medium values
- (c) changing `tol` in LinearSVC and LogisticRegression to high/low/medium values
- (d) changing the `penalty` (regularization) hyperparameter in LinearSVC and LogisticRegression (l2 vs l1)

You may of course perform and report all the above experiments and/or additional experiments not mentioned above (e.g. changing the `solver`, `max_iter` etc) but reporting at least 2 of the above experiments is mandatory. You do not need to submit code for these experiments – just report your findings in the PDF file. Your submitted code should only include two methods, one that maps raw challenges to embedded features, and another one that provides fixed/tuned values of chosen hyperparameters. (20 marks)

5. While the dataset provided to you has 7500 train CRPs, are those many needed? Show a plot of train size vs test accuracy. Perform at least 10 trials, each trial exploring a different train set size going at least as small as 100 train points and at least as large as 5000 train points. Do not change the test set for these different trials. For each trial, a smaller train set should be selected randomly (either with or without replacement) from within the original 7500-sized train set. Also comment on the minimum number of train CRPs needed to obtain 95% test accuracy, 97% test accuracy, 99% test accuracy? (20 marks)

Parts 1, 2, 4, 5 need to be answered in the PDF report file. Part 3 needs to be answered in the Python file.

Evaluation Measures and Marking Scheme We created two Hamming PUFs – a public one and a secret one. Both used 32-bit challenges but the secret words and thresholds in the secret Hamming PUF are different than those in the public Hamming PUF. Thus, a model that can predict the public Hamming PUF responses correctly is expected to do poorly at predicting the responses for the secret Hamming PUF and vice versa. The train/test sets we have provided you with the assignment package were created using CRPs from the public Hamming PUF. We similarly created a secret train and secret test set using CRPs from the secret Hamming PUF.

We will use your `my_map()` method to map challenges for the secret Hamming PUF dataset onto feature vectors and then learn a linearSVC model using the hyperparameter values provided by your `my_params()` method. The learnt model will be used to evaluate train and test accuracy on the secret dataset. Note that the secret train/test set may not have the same number of train/test points as the public train/test set that we have provided you. However, the public and secret Hamming PUFs were generated in a similar manner so it is likely that hyperparameter choices that seem good to you during validation (e.g. `C`, `tol`, `solver` etc), should work decently on the secret dataset as well. However, it is upto you whether you wish to provide fixed/preset hyperparameter values in the `my_params()` method (and get more marks for faster hyperparameter value generation) or else tune hyperparameters on the secret dataset (and possibly get lower marks for slower hyperparameter value generation but more assurance that the chosen hyperparameters will do well on the secret dataset). We will repeat the evaluation process described below 5 times and use the average performance so as to avoid any unluckiness

due to random choices inside your code in a particular trial. We will award marks based on the following performance parameters:

1. Time taken by your `my_map()` method to generate train + test features. (25 marks)
2. Dimensionality of your feature map i.e., how large is D . (20 marks)
3. Time taken by your `my_params()` method to return hyperparameter values. (10 marks)
4. Time taken by LinearSVC to train using your features and hyperparameters. (25 marks)

Once we receive your code, we will execute the validation script to award marks to your submission. For all the above performance parameters, lower is better (e.g. smaller D will get higher marks). For more details, please check the validation script on Google Colab (linked below). Marks will be calculated out of 80 as mentioned above and be multiplied by two *performance multipliers* – a train multiplier and a test multiplier that will be functions of the accuracy scores the trained model gets on the secret train and test datasets. For example, the multiplier could be $\eta = 10 \cdot \max(0, a - 0.9)$ where a is the train/test accuracy on the secret dataset. The final score would then be marks out of 80 calculated as above, multiplied with η_{train} and η_{test} . The exact formula for the multiplier will be announced along with the graded copies. The reason for the introduction of these multipliers is to penalize trivial submissions (e.g. identity map and default hyperparameters) that run super fast but give poor accuracy.

Validation on Google Colab. Before making a submission, you must validate your submission on Google Colab using the script linked below.

Link: <https://colab.research.google.com/drive/107xaxbukROKPFmV25F9TRzfkNn0fLYrk?usp=sharing>

Validation ensures that your submitted file `submit.py` does work with the automatic judge and does not give errors. Please use the IPYNB file at the above link on Google Colab and the dummy secret train, dummy secret test set and dummy model files (details below) to validate your submission.

Please make sure you do this validation on Google Colab itself. **Do not download the IPYNB file and execute it on your machine – instead, execute it on Google Colab itself.** This is because most errors we encounter are due to non-standard library versions on students' personal machines. Thus, running the IPYNB file on your personal machine defeats the whole purpose of validation. You must ensure that your submission runs on Google Colab to detect any library conflict. **Please note that there will be penalties for submissions which were not validated on Google Colab and which subsequently give errors with our automated judge.**

Dummy Submission File and Dummy Secret Files. In order to help you understand how we will evaluate your submission using the evaluation script, we have included a dummy secret train and secret test set in the assignment package itself (see the directory called `dummy`). However, note that these are just copies of the public train, test datasets we provided you. The reason for providing the dummy files is to allow you to check whether the evaluation script is working properly on Google Colab or not. Be warned that the secret files on which we actually evaluate your submission will be different. We have also included a dummy submission file `dummy_submit.py` to show you how your code must be written with `my_map()` and `my_params()` functions. Note that the choices made in `dummy_submit.py` are bad and give poor results. However, this is okay since its purpose is only to show you the code format.

Using Internet Resources. You are allowed to refer to textbooks, internet sources, research papers to find out more about this problem and for specific derivations e.g. the arbiter-PUF problem. However, if you do use any such resource, cite it in your PDF file. There is no penalty for using external resources with attribution but claiming someone else's work (e.g. a book or a research paper) as one's own work without crediting the original author will attract penalties.

Restrictions on Code Usage. You are allowed to use the numpy module in its entirety and any sklearn submodule that learns linear models and helper submodules e.g. calculate accuracy of a learnt model using `sklearn.metrics.accuracy_score`. However, **the use of non-linear models is prohibited** e.g. decision trees, random forests, nearest neighbors, neural networks, kernel SVMs etc. The use of other machine learning libraries such as **scipy, libsvm, keras, tensorflow is also prohibited**. Use of prohibited modules and libraries for whatever reason will result in penalties. For this assignment, you should also not download any code available online or use code written by persons outside your assignment group. Direct copying of code from online sources or amongst assignment groups will be considered an act of plagiarism for this major assignment and penalized according to pre-announced policies.

2 How to Prepare the PDF File

Use the following style file to prepare your report. The ZIP file contains an example file and instructions.

<https://media.neurips.cc/Conferences/NeurIPS2025/Styles.zip>

You must use the following command in the preamble

```
\usepackage[preprint]{neurips_2025}
```

instead of `\usepackage[preprint]{neurips_2025}`. Use proper L^AT_EX commands to neatly typeset your responses to the various parts of the problem. Use neat math expressions to typeset your derivations. Remember that all parts of the question need to be answered in the PDF file. All plots must be generated electronically - hand-drawn plots are unacceptable. All plots must have axes titles and a legend indicating what are the plotted quantities. Insert the plot into the PDF file using L^AT_EX `\includegraphics` commands.

3 How to Prepare the Python File

The major assignment package contains a skeleton file `submit_template.py` which you should fill in with the code of the method you think works best among the methods you tried. You must use this skeleton file to prepare your Python file submission (i.e. do not start writing code from scratch). This is because we will autograde your submitted code and so your code must have its input output behavior in a fixed format. Be careful not to change the way the skeleton file accepts input and returns output.

1. The skeleton code has comments placed to indicate non-editable regions. **Do not remove those comments.** We know they look ugly but we need them to remain in the code to keep demarcating non-editable regions.
2. The code file you submit should be self contained and should not rely on any other files (e.g. other .py files or else pickled files etc) to work properly. Remember, your ZIP archive should contain only one Python (.py) file. This means that you should store

any hyperparameters that you learn (e.g. C etc) inside that one Python file itself using variables/functions.

3. Certain portions of the skeleton code have been marked as non-editable. Please do not change these lines of code. Insert your own code within the designated areas only. If you tamper with non-editable code (for example, to make your code seem better or faster than it really is), the auto-grader may refuse to run your code and give you a zero instead (we will inspect each code file manually).
4. You are allowed to freely define new functions, new variables, new classes in inside your submission Python file while not changing the non-editable code.
5. The use of non-linear models is prohibited e.g. decision trees, random forests, nearest neighbors, neural networks, kernel SVMs etc. The use of other machine learning libraries such as scipy, libsvm, keras, tensorflow is also prohibited.
6. You are allowed to use basic Python libraries such as numpy, sklearn which have already been included for you.
7. Do take care to use broadcasted and array operations as much as possible and not rely on loops to do simple things like take dot products, calculate norms etc otherwise your solution will be slow and you may get less marks.
8. Please do not perform file operations in your code or access the disk – you should not be using libraries such as sys, pickle in your code.
9. Before submitting your code, make sure you validate on Google Colab to confirm that there are no errors etc.
10. You do not have to submit the evaluation script to us – we already have it with us. We have given you access to the Google Colab evaluation script just to show you how we would be evaluating your code and to also allow you to validate your code.